

Secure Modular Ktor Identity Provider Design Guide

12.08.2026 · adesso

Exported by Kilian Radtke

Secure Modular Ktor Identity Provider — Architecture and Implementation Concept

Deliverables and assumptions

I recommend building this as a **standards-first modular monolith** initially, with clear boundaries so protocol adapters and selected modules can later be extracted.

Default decisions

Area	Recommendation
Primary database	PostgreSQL
Cache / ephemeral state	Redis
Password hashing	Argon2id
Browser authentication	Secure opaque session cookie or BFF
SPA/native authentication	OAuth 2 Authorization Code + PKCE
Access tokens	Short-lived JWTs or opaque tokens; default to opaque where revocation is important
Refresh tokens	Rotating opaque tokens, stored only as hashes
MFA	TOTP first, WebAuthn/passkeys later
OAuth provider integration	Provider SPI with isolated adapters
Flow abstraction	Typed, explicit flow definitions with requirements, steps, transactions and effects
API errors	RFC 9457-style Problem Details with stable machine-readable <code>code</code>
Deployment	Stateless Ktor instances, PostgreSQL HA, Redis cluster, separate worker process
LDAP/SAML	Adapters, not domain-level special cases; defer SAML unless required

Important: An OAuth/OIDC authorization server is security-critical infrastructure. If identity is not a core product differentiator, use a proven identity server such as Keycloak, Zitadel, ORY, or another audited provider and place the Ktor application in front of it as a domain-specific façade. If you implement the authorization server yourself, keep the protocol surface narrow and use mature cryptographic libraries.

1. Architecture and module patterns

1.1 High-level architecture

Use a **hexagonal architecture** with a modular monolith as the first deployment unit.



Main layers

1. **Protocol/API layer**
 - Ktor routes.
 - OAuth2/OIDC endpoints.
 - Account-management endpoints.
 - DTO validation and error mapping.
 - Authentication middleware.
2. **Application/flow layer**
 - Registration.
 - Login.
 - MFA challenges.
 - Password changes.
 - Email changes.
 - OAuth linking.
 - Token issuance.
 - Session management.
3. **Domain layer**
 - Users.
 - Credentials.
 - External identities.
 - MFA factors.
 - Sessions.
 - Clients, scopes and consent.
 - Security policies.
 - Domain events.
4. **Infrastructure layer**
 - PostgreSQL repositories.
 - Redis rate limiting and short-lived state.
 - KMS/HSM integration.
 - Email delivery.
 - Provider HTTP clients.
 - Audit and telemetry.

1.2 Suggested Gradle module layout

identity-provider/

```

├── app/
│   └── Application.kt
├── auth-domain/
│   ├── user/
│   ├── credential/
│   ├── identity/
│   ├── session/
│   ├── mfa/
│   ├── passkey/
│   ├── client/
│   └── policy/
├── flow-engine/
│   ├── Flow.kt
│   ├── Requirement.kt
│   ├── Step.kt
│   ├── Effect.kt
│   └── FlowExecutor.kt
├── protocol-oauth2-oidc/
│   ├── AuthorizeEndpoint.kt
│   ├── TokenEndpoint.kt
│   ├── RevocationEndpoint.kt
│   ├── IntrospectionEndpoint.kt
│   ├── DiscoveryEndpoint.kt
│   └── JwksEndpoint.kt
├── provider-spi/
│   ├── ExternalIdentityProvider.kt
│   ├── CredentialSource.kt
│   └── ProviderProfile.kt
├── provider-google/
├── provider-github/
├── provider-discord/
├── provider-ldap/
├── mfa-totp/
├── mfa-webauthn/
├── persistence-postgres/
├── persistence-redis/
├── api-http/
├── audit-risk/
├── jobs/
└── test-support/

```

You do not have to create all Gradle modules immediately. Start with package boundaries and extract modules once the boundaries are stable.

1.3 Recommended separation of concepts

Do not model every authentication mechanism as the same kind of provider. Use separate abstractions:

```

sealed interface AuthSource {
    data object LocalPassword: AuthSource
    data object Passkey: AuthSource
    data object Ldap: AuthSource
}

```

```
data class ExternalOAuth(  
    val provider: ProviderKey  
): AuthSource  
}
```

External OAuth/OIDC provider

Responsible for:

- Authorization URL creation.
- State and PKCE handling.
- Code exchange.
- ID-token validation.
- User-profile retrieval.
- Provider-specific normalization.

```
interface ExternalIdentityProvider {  
    val key: ProviderKey  
  
    suspend fun createAuthorizationRequest(  
        request: ProviderAuthorizationRequest  
    ): AuthorizationRedirect  
  
    suspend fun exchangeCode(  
        request: ProviderCallbackRequest  
    ): ExternalProfile  
}
```

Credential source

LDAP is different from Google or GitHub. LDAP authenticates credentials against a directory and may expose groups or attributes.

```
interface CredentialSource {  
    val key: CredentialSourceKey  
  
    suspend fun authenticate(  
        identifier: String,  
        secret: Secret  
    ): CredentialAuthenticationResult  
}
```

MFA method

```
interface MfaMethod {  
    val type: MfaType  
  
    suspend fun beginEnrollment(  
        userId: UserId  
    ): EnrollmentChallenge  
  
    suspend fun verify(  
        challenge: MfaChallenge,  
        response: MfaResponse  
    ): Boolean  
}
```

```
); VerificationResult  
}
```

New MFA methods should implement this interface without changing registration or login domain logic.

1.4 Core design principle

The API route should not contain security logic such as:

- Checking whether MFA is required.
- Deciding whether reauthentication is needed.
- Issuing refresh tokens.
- Updating user state.
- Linking an external identity.
- Sending security emails.

The route should only:

5. Parse and validate the request.
 6. Build a command.
 7. Execute a flow.
 8. Map the typed result to HTTP.
-

2. Flow engine and reusable flow builder

2.1 What the flow engine should solve

The flow engine should provide:

- Explicit preconditions.
- Reauthentication requirements.
- MFA requirements.
- Typed commands and results.
- Transaction boundaries.
- Idempotency.
- Consistent failure mapping.
- Audit hooks.
- Transactional outbox effects.
- Retry and replay protection.
- Testability without Ktor.

It should **not** become a generic workflow engine for arbitrary business processes. Keep it focused on security-sensitive identity flows.

2.2 Flow concepts

Context

```
data class FlowContext(  
    val requestId: RequestId,  
    val clientId: ClientId?,  
    val actor: Principal?,
```

```
val ipAddress: InetAddress?,  
val userAgent: String?,  
val idempotencyKey: String?,  
val now: Instant  
)
```

Result

```
sealed interface FlowResult<out R> {  
    data class Success<R>(  
        val value: R  
    ): FlowResult<R>  
  
    data class Challenge(  
        val code: String,  
        val transactionId: String,  
        val expiresAt: Instant,  
        val methods: Set<String> = emptySet()  
    ): FlowResult<Nothing>  
  
    data class Failure(  
        val error: AuthError  
    ): FlowResult<Nothing>  
}
```

Requirement

Requirements should produce explicit outcomes, not plain Boolean values.

```
interface Requirement<C> {  
    suspend fun check(  
        command: C,  
        context: FlowContext  
    ): RequirementResult  
}  
  
sealed interface RequirementResult {  
    data object Satisfied: RequirementResult  
  
    data class Rejected(  
        val error: AuthError  
    ): RequirementResult  
  
    data class ChallengeRequired(  
        val challenge: ChallengeDescriptor  
    ): RequirementResult  
}
```

Examples:

- `Authenticated`
- `ReauthenticatedWithin(5.minutes)`
- `EmailVerified`

- MfaSatisfied
- HasPermission("account:password:change")
- CurrentPasswordValid
- ClientAllowedForScope
- ValidOAuthState
- ValidPkceVerifier

2.3 Typed flow DSL

A practical flow DSL could look like this:

```
interface FlowDsl<C, R> {
    fun require(requirement: Requirement<C>)
    fun step(step: FlowStep<C>)
    fun transaction(policy: TransactionPolicy)
    fun commitAction(action: CommitAction<C>)
    fun effect(effect: DomainEffect<C>)
    fun onFailure(handler: FailureHandler<C>)
}

interface FlowStep<C> {
    suspend fun execute(
        command: C,
        context: FlowContext,
        state: MutableFlowState
    ): StepResult
}
```

Flow definition:

```
val changePasswordFlow =
    flow<ChangePasswordCommand, Unit>(
        id = FlowId("account.change-password")
    ) {
        transaction(TransactionPolicy.Required)

        require(Authenticated)
        require(ReauthenticatedWithin(5.minutes))
        require(EmailVerified)

        step(ValidateCurrentPassword())
        step(ValidateNewPasswordPolicy())
        step(HashNewPassword())
        step(StorePassword())

        commitAction(RevokeOtherSessions())
        effect>PasswordChangedNotification()
        effect>PasswordChangedAuditEvent()
    }
```

The exact syntax is less important than these properties:

- Requirements are visible in the flow definition.

- Steps are deterministic and testable.
- Database updates are transactional.
- External effects execute after commit.
- Sensitive challenges are short-lived and single-use.

2.4 Transaction and side-effect rules

Inside the database transaction

- User state changes.
- Credential changes.
- Token creation or revocation.
- Session revocation.
- Identity linking.
- Audit event creation.
- Outbox event creation.

After commit

- Email delivery.
- Webhook delivery.
- Push notifications.
- Analytics events.
- Cache invalidation.
- Provider synchronization.

Never send an email or call an external provider inside the transaction that updates the user. If the network call fails, the database transaction should not be rolled back unnecessarily.

2.5 Idempotency

Require idempotency keys for operations that can be retried:

- Registration.
- Password reset completion.
- Email change completion.
- OAuth callback processing.
- Provider linking.
- MFA enrollment completion.

Store an idempotency record containing:

```
idempotency_key
client_id
user_id
request_hash
status
response_hash
expires_at
```

For OAuth callbacks, the authorization code and `state` must additionally be single-use.

2.6 Typical flow behavior

Registration

9. Rate-limit by IP, email and username.
10. Normalize username and email.
11. Validate password policy and breached-password list.
12. Create user in `PENDING_EMAIL_VERIFICATION`.
13. Hash password with Argon2id.
14. Create one-time email verification token.
15. Write an outbox event.
16. Return a generic `202 Accepted`.
17. Never reveal whether the email already exists.

Login

18. Rate-limit before expensive password verification.
19. Load the account using normalized identifiers.
20. Use a dummy Argon2id verification path if the account does not exist.
21. Verify the password.
22. Check account status.
23. Check whether email verification is required.
24. Check MFA policy.
25. If MFA is required, create a short-lived MFA transaction.
26. Otherwise issue a session or authorization code.
27. Write an audit event.

Password reset

28. Always return the same response for an existing or unknown email.
29. Create a random 256-bit reset token.
30. Store only its hash.
31. Use a 15-minute expiry.
32. Invalidate previous reset tokens for the same account.
33. On completion:
 - Verify the token.
 - Replace the password hash.
 - Revoke refresh-token families.
 - Revoke other sessions.
 - Send a security notification.
7. Do not automatically trust a password-reset session as equivalent to MFA.

Change email

8. Require an authenticated session.
9. Require recent reauthentication.
10. Require MFA if configured.
11. Store the new address as pending.

12. Send verification to the new address.
13. Notify the old address.
14. Commit only after successful verification.
15. Consider revoking sessions if the risk engine detects a suspicious change.

Link OAuth provider

16. Require authentication.
17. Require recent reauthentication.
18. Require MFA for high-risk accounts.
19. Start provider authorization with:
 - state
 - PKCE
 - nonce for OIDC
5. Validate issuer, subject, nonce, code and redirect.
6. Link by stable (issuer, subject), never by email alone.
7. Notify the account owner.
8. Do not silently merge accounts.

Unlink OAuth provider

Allow unlinking only when another secure authentication method remains:

- Local password.
 - Another external identity.
 - Passkey.
 - MFA recovery path.
-

3. Data model and persistence

3.1 Why PostgreSQL

PostgreSQL is the preferred system of record because it provides:

- Strong unique constraints.
- Transactions across users, identities and tokens.
- Row-level locking.
- Good support for audit and outbox patterns.
- Mature backup and replication.
- JSONB for provider-specific metadata.
- Strong indexing and query planning.
- Excellent operational tooling.

A document database is possible, but it makes the following more difficult:

- Preventing duplicate external identities.
- Enforcing unique usernames and emails.
- Atomic refresh-token rotation.
- Linking identities safely.
- Maintaining audit and outbox consistency.

Use Redis for fast-changing, short-lived data. Do not use Redis as the only source of truth for security state.

3.2 Recommended entities

users

```
id
username
username_normalized
primary_email
primary_email_normalized
email_verified_at
status
created_at
updated_at
version
```

Recommended statuses:

```
PENDING_EMAIL_VERIFICATION
ACTIVE
SUSPENDED
LOCKED
DELETED
```

password_credentials

```
user_id
password_hash
algorithm
parameters_json
changed_at
created_at
```

Do not place the password hash directly in the user table if you expect multiple credential types.

external_identities

```
id
user_id
provider_key
issuer
subject
provider_email
profile_json
created_at
last_login_at
```

Required unique constraint:

```
UNIQUE (issuer, subject)
```

The stable identity key is the provider issuer plus provider subject.

mfa_factors

```
id
user_id
type
label
status
created_at
last_used_at
```

totp_factors

```
factor_id
encrypted_secret
secret_key_version
algorithm
digits
period_seconds
last_accepted_step
created_at
```

TOTP secrets must be encrypted, not hashed, because the service needs to verify future codes.

recovery_codes

```
id
user_id
code_hash
used_at
created_at
```

Store recovery codes as hashes. Show them once during enrollment.

webauthn_credentials

```
id
user_id
credential_id
public_key
signature_counter
aaguid
transports
user_verified_required
backup_eligible
backup_state
created_at
last_used_at
```

Never store a passkey private key.

email_verification_tokens

```
id
user_id
token_hash
purpose
expires_at
used_at
created_at
```

password_reset_tokens

Same structure, with a separate purpose.

sessions

```
id
user_id
session_hash
client_id
created_at
last_seen_at
idle_expires_at
absolute_expires_at
revoked_at
ip_hash
user_agent_hash
```

refresh_token_families

```
id
user_id
client_id
created_at
absolute_expires_at
revoked_at
reuse_detected_at
```

refresh_tokens

```
id
family_id
token_hash
issued_at
expires_at
used_at
revoked_at
replaced_by_token_id
```

OAuth/OIDC entities

```
oauth_clients
oauth_redirect_uris
```

```
oauth_client_scopes
authorization_codes
consents
signing_keys
```

Authorization codes should be short-lived, single-use and stored as hashes.

audit_logs

```
id
event_type
actor_user_id
subject_user_id
client_id
request_id
ip_hash
user_agent_hash
metadata_json
created_at
```

Do not log:

- Passwords.
- Tokens.
- Authorization codes.
- TOTP values.
- Recovery codes.
- Full provider responses.
- Raw cookies.

outbox_events

```
id
event_type
aggregate_type
aggregate_id
payload_json
attempts
available_at
processed_at
last_error
created_at
```

3.3 Indexes and constraints

At minimum:

```
UNIQUE users.username_normalized
UNIQUE users.primary_email_normalized
UNIQUE external_identities(issuer, subject)
INDEX sessions(user_id, revoked_at)
INDEX refresh_token_families(user_id, revoked_at)
INDEX token_tables(expires_at)
INDEX audit_logs(subject_user_id, created_at)
```

INDEX outbox_events(processed_at, available_at)

Use case-folding and Unicode normalization consistently. Define the normalization policy centrally; do not let individual routes normalize differently.

3.4 Migration strategy

Use Flyway or Liquibase.

Recommended rules:

9. Every schema change is versioned.
 10. Migrations are applied automatically during deployment only if your release process supports safe gating.
 11. Prefer expand-and-contract migrations:
 - Add nullable column.
 - Deploy code that writes both representations.
 - Backfill.
 - Switch reads.
 - Remove old representation later.
 4. Never delete security data immediately. Use retention and soft deletion where required.
 5. Test migrations against a production-like PostgreSQL version.
 6. Maintain a rollback or forward-fix strategy.
-

4. Security and best practices

4.1 Password hashing

Preferred

Argon2id with deployment-specific benchmarking.

A reasonable starting point:

Parameter	Starting value
Memory	64 MiB
Iterations	3
Parallelism	1-2
Salt	16 random bytes
Output	32 bytes
Target duration	Approximately 100-300 ms per verification

Tune the values under realistic concurrency. Password verification must not exhaust the application thread pool.

Fallback

If Argon2id is not available:

- bcrypt cost 12-14.
- Explicitly handle bcrypt's 72-byte input limit.
- Normalize and pre-hash long passwords carefully.
- Do not silently truncate passwords.

Use:

- Minimum length around 12 characters.
- Maximum length at least 64 characters.
- Breached-password checks.
- No mandatory composition rules unless required by policy.
- Password history only if there is a strong business requirement.

A server-side pepper can be added, but it must be stored in KMS/HSM, not in the database.

4.2 Access-token strategy

Recommended default

For subsidiary APIs with strict revocation requirements:

- Opaque access tokens.
- Introspection endpoint.
- Short cache duration.
- Centralized revocation.

For high-scale service-to-service APIs:

- Short-lived asymmetric JWT access tokens.
- `EdDSA` or `ES256` preferred.
- RSA only where ecosystem compatibility requires it.
- Never distribute a shared symmetric signing secret to every subsidiary.

Recommended JWT claims:

```
iss
sub
aud
exp
iat
jti
scope
client_id
amr
acr
```

Do not include unnecessary personal data in access tokens.

Suggested lifetimes

Credential	Recommended lifetime
Authorization code	60 seconds
Access token	5–10 minutes
ID token	5 minutes
Browser session idle timeout	8–12 hours
Browser session absolute timeout	7–30 days
Refresh token inactivity timeout	30 days
Refresh token absolute timeout	90 days
MFA transaction	5 minutes
Email verification token	24 hours
Password reset token	15 minutes

4.3 Refresh-token rotation

Use a token family model.

On every refresh:

7. Hash and locate the presented refresh token.
8. Verify that it is active.
9. Mark it as used.
10. Create a replacement token.
11. Return the replacement token.
12. If a previously used token is presented again:
 - Mark the family as compromised.
 - Revoke all tokens in the family.
 - Require reauthentication.

Store only the hash:

```
HMAC-SHA-256(server_pepper, refresh_token)
```

or a cryptographic hash of a high-entropy random token.

Native clients should store refresh tokens in:

- iOS Keychain.
- Android Keystore-backed storage.
- Equivalent OS-protected secure storage.

Never store long-lived refresh tokens in browser `localStorage`.

4.4 Cookies and browser sessions

For a browser session:

```
Name: __Host-idp_session  
Secure: true  
HttpOnly: true  
SameSite: Lax or Strict  
Path: /  
Domain: omitted
```

The `__Host-` prefix requires:

- `Secure`.
- `No Domain`.
- `Path=`.

For SPAs, the preferred architecture is a **Backend-for-Frontend**:

```
Browser -> BFF cookie session -> Ktor BFF -> OAuth/OIDC provider
```

This avoids exposing long-lived tokens to browser JavaScript.

If a SPA communicates directly with the authorization server:

- Use Authorization Code + PKCE.
- Keep access tokens in memory where possible.
- Use a secure refresh-token cookie only if CSRF protections are complete.
- Never use the implicit flow.

- Never use the password grant.

4.5 CSRF protection

Cookie-authenticated state-changing endpoints require CSRF protection.

Use multiple defenses:

13. SameSite=Lax or Strict.
14. Verify Origin.
15. Verify Referer where appropriate.
16. Use a synchronizer token or double-submit token.
17. Require JSON content types where appropriate.
18. Reject unexpected Sec-Fetch-Site values as defense in depth.

Bearer-token APIs are generally not vulnerable to classic cookie CSRF, but they remain vulnerable to XSS and token theft.

4.6 Rate limiting and brute-force protection

Use Redis or another distributed limiter.

Apply limits by multiple dimensions:

- IP address.
- Account identifier.
- Client ID.
- Device/session fingerprint.
- Provider.
- Endpoint.

Example starting limits:

Operation	Suggested limit
Login by account and IP	5 attempts / 10 minutes
Login by IP	20 attempts / 10 minutes
Password reset request	3 / hour per email and IP
TOTP verification	5 / 5 minutes
Email verification	5 / 10 minutes
OAuth callback failures	20 / 10 minutes per client and IP
Token endpoint	Per client and IP

Do not use permanent account lockouts as the only control. They enable denial of service. Prefer:

- Progressive delays.
- Temporary risk challenges.
- CAPTCHA only when risk is elevated.
- MFA step-up.
- Notifications.
- Account recovery workflows.

4.7 TOTP implementation

Recommended defaults:

- Random secret: at least 160 bits.
- 6-digit codes.
- 30-second time step.

- HMAC-SHA-1 for broad compatibility, or SHA-256 if all clients support it.
- Verification window: ± 1 step.
- One accepted code per time step.
- Encrypted secret storage.
- KMS-managed encryption key.
- Versioned encryption metadata.

Enrollment:

19. Require authenticated session.
20. Require recent reauthentication.
21. Generate secret.
22. Display QR code and manual secret once.
23. Require a valid TOTP code to confirm enrollment.
24. Generate recovery codes.
25. Store the factor as active only after successful confirmation.
26. Audit the event.

Do not display the TOTP secret again after enrollment.

4.8 WebAuthn/passkeys

Store:

- Credential ID.
- Public key.
- Signature counter.
- User ID.
- RP ID.
- AAGUID.
- Transport hints.
- Backup eligibility and state.
- User-verification policy.

The server must validate:

- Challenge.
- Origin.
- RP ID.
- User handle.
- Signature.
- Sign counter behavior.
- User presence.
- User verification when required.

Attestation:

- Use none or indirect attestation by default.
- Require direct or enterprise attestation only when you have a concrete device-trust requirement.
- Maintain an allowlist policy if regulated environments require specific authenticators.

Challenges must be:

- Random.
- Stored server-side or in a trusted short-lived store.
- Single-use.

- Expired within a few minutes.
- Bound to the intended operation.

4.9 OAuth/OIDC SSRF and callback protection

Do not allow arbitrary provider URLs from client input.

For provider adapters:

- Use a configured provider registry.
- Allowlist issuer URLs.
- Require HTTPS outside local development.
- Validate discovery metadata.
- Validate issuer equality.
- Disable unsafe redirects.
- Use strict HTTP timeouts.
- Block private and link-local IP ranges.
- Prevent DNS rebinding.
- Prefer an egress proxy for outbound provider traffic.
- Never fetch user-controlled URLs from provider profile fields.
- Use exact redirect URI matching.
- Validate `state`, `nonce`, PKCE verifier, issuer, audience and subject.

4.10 Account-takeover protections

Require step-up authentication for:

- Password change.
- Email change.
- Provider linking.
- Provider unlinking.
- MFA disabling.
- Recovery-code regeneration.
- Trusted-device changes.
- Suspicious new-device login.

Send notifications for:

- New login.
 - Password change.
 - Email change.
 - Provider linked or unlinked.
 - MFA enabled or disabled.
 - Refresh-token reuse.
 - Recovery-code usage.
-

5. API design and error conventions

5.1 Protocol endpoints

Implement standards-compliant endpoints:

`GET /.well-known/openid-configuration`

```
GET /.well-known/jwks.json
```

```
GET /oauth2/authorize
```

```
POST /oauth2/token
```

```
POST /oauth2/revoke
```

```
POST /oauth2/introspect
```

For OIDC:

```
GET /userinfo
```

Authorization Code + PKCE should be the primary client flow.

Do not implement the Resource Owner Password Credentials grant.

Example authorization request

```
GET /oauth2/authorize
?response_type=code
&client_id=subsidiary-spa
&redirect_uri=https%3A%2F%2Fapp.example%2Fcallback
&scope=openid%20profile%20email
&state=...
&nonce=...
&code_challenge=...
&code_challenge_method=S256
```

Client categories

Client	Recommended flow
Server-rendered web app	Authorization Code
SPA	Authorization Code + PKCE, preferably through a BFF
Native mobile app	Authorization Code + PKCE
Confidential backend	Authorization Code, client secret or private-key JWT
Service-to-service	Client Credentials with strong client authentication

5.2 Account API

Use a separate versioned API:

```
POST /v1/auth/register
POST /v1/auth/login # first-party/BFF only
POST /v1/auth/logout
POST /v1/auth/password-reset/request
POST /v1/auth/password-reset/complete
POST /v1/auth/email/verify
POST /v1/auth/email/resend

GET /v1/me
PUT /v1/me/password
POST /v1/me/email-change
POST /v1/me/mfa/totp/enroll
POST /v1/me/mfa/totp/verify
```

```
DELETE /v1/me/mfa/totp
POST /v1/me/providers/{provider}/link
DELETE /v1/me/providers/{provider}
GET /v1/me/sessions
DELETE /v1/me/sessions/{sessionId}
```

Do not use `/v1/auth/login` as a replacement for OAuth authorization. It should be restricted to first-party flows such as a BFF.

5.3 Standard error format

Use `application/problem+json`.

```
{
  "type": "https://identity.example/errors/mfa-required",
  "title": "Additional authentication required",
  "status": 401,
  "code": "mfa_required",
  "detail": "Complete the additional authentication step.",
  "request_id": "req_01J...",
  "transaction_id": "mfa_tx_01J...",
  "methods": [
    "totp",
    "webauthn"
  ],
  "expires_at": "2026-08-12T14:30:00Z"
}
```

The following fields should be stable:

```
type
title
status
code
request_id
transaction_id
retry_after
details
```

Do not use `detail` to reveal sensitive information.

5.4 Recommended error codes

Authentication

```
auth_required
invalid_credentials
invalid_token
token_expired
token_revoked
reauthentication_required
mfa_required
mfa_invalid
mfa_expired
```

Account state

email_unverified
account_suspended
account_locked
account_disabled
password_reset_required

OAuth/provider

provider_unavailable
provider_denied
provider_invalid_response
provider_link_required
provider_already_linked
identity_already_linked
oauth_state_invalid
oauth_nonce_invalid
pkce_verifier_invalid
redirect_uri_invalid

Validation and infrastructure

validation_failed
conflict
not_found
forbidden
rate_limited
temporarily_unavailable
idempotency_conflict

5.5 HTTP mapping

Condition	Status
Authentication required	401
Invalid credentials	401
MFA required	401
Invalid bearer token	401
Email unverified	403
Account suspended	403
Permission denied	403
Provider linking required	409 or 403, depending on flow
Validation failure	422
Duplicate resource	409
Rate limited	429
Temporary provider failure	503

For bearer authentication, also send:

WWW-Authenticate: Bearer realm="identity", error="invalid_token"

5.6 Frontend behavior

A small client library can map codes deterministically:

```
auth_required
-> clear local session and navigate to login

mfa_required
-> open MFA transaction UI

email_unverified
-> show verification and resend action

token_expired
-> attempt one refresh, then sign out

rate_limited
-> honor Retry-After and apply backoff

provider_link_required
-> start authenticated provider-linking flow
```

Avoid making frontend code depend on human-readable text.

6. Integrations and libraries

6.1 Ktor plugins

Recommended Ktor components:

- ktor-server-auth
- ktor-server-auth-jwt for JWT verification where needed
- ktor-server-content-negotiation
- ktor-server-status-pages
- ktor-server-call-id
- ktor-server-call-logging
- ktor-server-request-validation
- ktor-server-cors
- ktor-server-forwarded-header only with a trusted proxy configuration
- ktor-server-metrics-micrometer
- Ktor client ContentNegotiation
- Ktor client HttpTimeout
- Ktor client HttpRequestRetry, used conservatively
- Ktor client TLS configuration
- kotlinx.serialization

Do not assume the Ktor JWT plugin is a complete authorization-server implementation. It is mainly useful for token verification.

6.2 Cryptography and tokens

Suitable JVM options:

- Nimbus JOSE + JWT.
- jose4j.
- Bouncy Castle where required.
- Java Cryptography Architecture for standard primitives.
- Argon2id implementation with audited native or JVM bindings.
- KMS/HSM SDK for key protection.

Do not implement:

- JWT signing.
- JWS validation.
- PKCE hashing.
- TOTP algorithms.
- WebAuthn signature verification.
- Argon2id.
- Password reset token generation.

6.3 OAuth/OIDC providers

For Google, GitHub and Discord:

- Use a common provider SPI.
- Use Ktor HTTP client for transport.
- Use Nimbus or another mature JOSE library for OIDC tokens.
- Keep provider-specific profile mapping in the adapter.
- Treat GitHub as a provider API integration rather than assuming every provider behaves like OIDC.
- Verify provider documentation and scopes during implementation.

6.4 WebAuthn

Evaluate:

- WebAuthn4J.
- Yubico WebAuthn server libraries.

Do not write WebAuthn verification logic yourself.

6.5 TOTP

A JVM TOTP library such as `dev.samstevens.totp` can be used, provided:

- The version is actively maintained.
- The library is reviewed for secure randomness.
- Verification-window behavior is understood.
- Secret storage is implemented by your application, not delegated blindly to the library.

6.6 Persistence

Recommended

- PostgreSQL.
- HikariCP.
- Flyway.
- JDBI, jOOQ or Exposed.

Recommendation between JDBI, Exposed and JPA

Option	Recommendation
JDBI	Strong choice when you want explicit SQL and low abstraction
jOOQ	Strong choice for SQL-heavy systems and generated types; check licensing
Exposed	Good Kotlin integration and acceptable for many applications
JPA/Hibernate	Usable, but avoid implicit behavior in the security core

For this system, I would use:

- **JDBI or jOOQ** for the protocol and security-critical persistence.
- **Exposed** if the team is already highly productive with it.
- Avoid introducing JPA solely because it is familiar.

6.7 Redis

Use Redis for:

- Rate limits.
- OAuth state.
- PKCE transactions.
- MFA transactions.
- Distributed locks.
- Short-lived authorization state.
- Cache invalidation.

Use PostgreSQL for:

- Users.
- Credentials.
- Sessions.
- Refresh-token families.
- Audit records.
- Outbox events.
- Consent and client configuration.

7. Deployment and operations

7.1 Secrets management

Use a dedicated secret manager:

- HashiCorp Vault.
- Azure Key Vault.
- AWS KMS/Secrets Manager.

- Google Cloud KMS/Secret Manager.
- Cloud HSM where required.

Secrets include:

- Database credentials.
- OAuth client secrets.
- Email provider credentials.
- Encryption keys.
- JWT signing keys.
- Password pepper.
- Session or token HMAC keys.

Never keep production secrets in:

- Git.
- Container images.
- Helm values committed to source control.
- Application logs.
- Crash reports.

7.2 Key rotation

Use key IDs:

```
kid = auth-signing-key-2026-08
```

For signing keys:

27. Generate a new key.
28. Publish the new public key in JWKS.
29. Start signing with the new key.
30. Keep the old public key available until all tokens expire.
31. Retire the old key.
32. Maintain an emergency revocation process.

For encrypted TOTP secrets:

- Store the encryption-key version beside the ciphertext.
- Support decrypting old versions.
- Re-encrypt on read or during a controlled migration.

7.3 Horizontal scaling

Keep Ktor instances stateless.

Do not store authentication state only in local memory.

Use:

- PostgreSQL for durable state.
- Redis for short-lived distributed state.
- Outbox workers for asynchronous effects.
- Load balancer health checks.
- Connection-pool limits.
- Database back-pressure.
- Circuit breakers for provider integrations.

7.4 CORS

Use explicit allowlists.

Do not use:

```
Access-Control-Allow-Origin: *  
Access-Control-Allow-Credentials: true
```

For each client:

- Configure exact origins.
- Configure exact methods.
- Configure exact headers.
- Expose only required response headers.
- Keep credentials disabled unless explicitly needed.

7.5 Logging and audit

Use structured logs with:

```
timestamp  
request_id  
trace_id  
event_type  
client_id  
user_id  
outcome  
provider  
latency
```

Do not include:

- Passwords.
- Bearer tokens.
- Cookies.
- TOTP codes.
- Reset tokens.
- Authorization codes.
- Full OAuth responses.

Useful metrics:

```
auth_login_attempts_total  
auth_login_failures_total  
auth_mfa_challenges_total  
auth_refresh_reuse_detected_total  
oauth_callback_failures_total  
password_hash_duration_seconds  
provider_latency_seconds  
rate_limit_rejections_total  
outbox_delivery_failures_total
```

Do not use raw email addresses or usernames as high-cardinality metric labels.

7.6 Availability and failure behavior

Fail closed for:

- Token validation failures.
- Key-store failures.
- Client validation failures.

- Unknown redirect URIs.
- MFA verification failures.
- Provider issuer mismatches.

For non-critical services:

- Email provider outages should not corrupt the user transaction.
 - Provider profile enrichment should not block local identity lookup indefinitely.
 - Webhooks should retry through the outbox.
 - Rate limiting should degrade conservatively, not silently disable protection.
-

8. Kotlin/Ktor code sketches

These are intentionally abbreviated design sketches, not production-ready code.

8.1 Route builder and flow execution

```
class RouteBuilder<T: AuthSource>(  
    private val source: T,  
    private val flowRunner: FlowRunner  
) {  
    fun <C: Any, R: Any> execute(  
        flow: Flow<C, R>,  
        commandFactory: suspend ApplicationCall.() -> C  
    ): RouteHandler = {  
        val command = commandFactory()  
        val context = call.toFlowContext(source)  
  
        val result = flowRunner.execute(  
            flow = flow,  
            command = command,  
            context = context  
        )  
  
        call.respond(result.toHttpResponse())  
    }  
}
```

A route should remain explicit:

```
fun Route.accountRoutes(  
    flowRunner: FlowRunner  
) {  
    put("/v1/me/password") {  
        val command = call.receive<ChangePasswordCommand>()  
  
        val result = flowRunner.execute(  
            flow = changePasswordFlow,  
            command = command,  
            context = call.toFlowContext(AuthSource.LocalPassword)  
        )  
    }  
}
```

```
)

call.respond(result.toHttpResponse())
}
}
```

The route builder may reduce repeated wiring, but it should not hide security-critical policy.

8.2 Change-password flow

```
data class ChangePasswordCommand(
    val currentPassword: Secret,
    val newPassword: Secret
)

val changePasswordFlow =
    flow<ChangePasswordCommand, Unit>{
        id = FlowId("account.change-password")
    } {
        transaction(TransactionPolicy.Required)

        require(Authenticated)
        require(ReauthenticatedWithin(Duration.ofMinutes(5)))
        require(EmailVerified)

        step(ValidateCurrentPassword())
        step(ValidateNewPasswordPolicy())
        step(HashPassword())
        step(StorePassword())

        commitAction(RevokeOtherSessions())
        effect>PasswordChangedAuditEvent()
        effect>PasswordChangedNotification()
    }
}
```

Important properties:

- `Secret` should prevent accidental logging.
- The current password is verified inside the flow.
- The new password is never stored in the flow context after hashing.
- Session revocation is committed atomically.
- Notifications execute through the outbox.

8.3 Example Exposed-style entities

```
object Users: UUIDTable("users") {
    val username = varchar("username", 128)
    val usernameNormalized = varchar("username_normalized", 128)
        .uniqueIndex()

    val primaryEmail = varchar("primary_email", 320)
    val primaryEmailNormalized = varchar("primary_email_normalized", 320)
```

```

.uniqueIndex()

val status = enumerationByName(
    "status",
    40,
    UserStatus::class
)

val emailVerifiedAt = timestampWithTimeZone("email_verified_at")
.nullable()

val createdAt = timestampWithTimeZone("created_at")
val updatedAt = timestampWithTimeZone("updated_at")
}

object PasswordCredentials: Table("password_credentials") {
    val userId = uuid("user_id")
    .references(Users.id)
    .uniqueIndex()

    val passwordHash = text("password_hash")
    val algorithm = varchar("algorithm", 32)
    val parameters = jsonb("parameters")
    val changedAt = timestampWithTimeZone("changed_at")
}

```

Password requirement clarification

Your product requirement says every user must have:

- Username.
- Password.
- Email.

That is valid for the initial local-account model. However, OAuth-only or passkey-first accounts may not have a local password. Do not create a fake password hash for such users.

Use one of these policies:

33. **Strict local-password policy:** every account must set a local password before activation.
34. **Credential-flexible policy:** local password is mandatory only for local accounts; external-only accounts use another credential source.

I recommend option 2 internally, while enforcing option 1 if your business domain explicitly requires a password for every account.

8.4 TOTP verification endpoint

```

@Serializable
data class TotpVerifyRequest(
    val transactionId: String,
    val code: String
)

```

```

post("/v1/auth/mfa/totp/verify") {
    val request = call.receive<TotpVerifyRequest>()

    val command = VerifyTotpCommand(
        transactionId = request.transactionId,
        code = request.code,
        requestId = call.requestId()
    )

    val result = flowRunner.execute(
        flow = verifyTotpFlow,
        command = command,
        context = call.toFlowContext(AuthSource.LocalPassword)
    )

    call.respond(result.toHttpResponse())
}

```

The flow should:

35. Load the MFA transaction.
36. Confirm it is unexpired and unused.
37. Load the encrypted TOTP secret.
38. Decrypt through KMS-backed key material.
39. Verify the code using the configured time window.
40. Reject a previously accepted time step.
41. Mark the challenge as completed.
42. Issue the pending session or authorization code.
43. Write an audit event.

Do not pass the decrypted secret through logs, generic flow metadata, or serialized state.

9. Risks and trade-offs

9.1 Building an authorization server yourself

This is the largest risk.

The difficult parts are not only password hashing or JWT signing. They include:

- Redirect URI validation.
- Consent and scopes.
- PKCE.
- Refresh-token replay.
- OIDC nonce handling.
- Client authentication.
- Session and logout semantics.
- Key rotation.
- Provider federation.
- Account linking.
- Recovery and takeover resistance.

- Back-channel effects.
- Browser security behavior.

If identity is not the product itself, strongly consider using a proven authorization server and implementing only the domain-specific account API in Ktor.

9.2 JWT versus opaque access tokens

JWT advantages

- No introspection call for every request.
- Good performance.
- Easy horizontal scaling.
- Works well across subsidiaries.

JWT disadvantages

- Revocation is harder.
- Claims remain valid until expiry.
- Key rotation and audience management are operationally important.
- Excessive claims create privacy and coupling problems.

Opaque-token advantages

- Central revocation.
- Smaller client exposure.
- Easier policy changes.
- Better control over session state.

Opaque-token disadvantages

- Introspection traffic.
- Cache invalidation.
- More dependency on the authorization server.

A hybrid architecture is reasonable:

- Opaque browser sessions.
- Short-lived JWTs for high-volume subsidiary APIs.
- Rotating opaque refresh tokens in both cases.

9.3 Flow-engine over-abstraction

A generic builder can become harder to understand than explicit code.

Avoid:

- Dynamically composed security policies from configuration.
- Reflection-heavy step discovery.
- Hidden transaction boundaries.
- Generic steps with unclear side effects.
- Route definitions that make authorization invisible.

Prefer named, typed flows with explicit requirements.

9.4 OAuth account linking

Automatic account linking by email is dangerous because:

- Provider emails may be unverified.
- Email ownership policies differ.
- An attacker may control an email claim.
- Multiple providers may represent the same human differently.

Link based on a stable provider identity, and require the existing user to authenticate before linking.

9.5 LDAP

LDAP introduces:

- Directory availability dependencies.
- Group-to-role mapping.
- Password policy differences.
- Connection and timeout management.
- TLS certificate validation.
- Directory-specific failure behavior.

Do not store LDAP passwords. Authenticate through the directory adapter and store only the mapped local identity and authorization metadata.

9.6 SAML

SAML is significantly more complex than OAuth/OIDC:

- XML signature validation.
- Metadata handling.
- Certificate rollover.
- Clock skew.
- Assertion replay.
- Audience restrictions.
- Attribute mapping.

Defer SAML unless there is a concrete enterprise requirement. When needed, use a mature library and keep SAML-specific logic isolated in a federation module.

9.7 Multi-tenancy

If subsidiaries require tenant isolation, decide early whether:

- One global user namespace is sufficient.
- Users can belong to multiple organizations.
- Roles and scopes are tenant-specific.
- Clients are owned by tenants.
- External identities are global or tenant-scoped.

Do not retrofit tenancy into token claims and database constraints later without a clear model.

10. Prioritized roadmap

Phase 0 — Security foundation

Implement first:

- Threat model.
- Data classification.
- Secure configuration.
- PostgreSQL schema.
- Flyway migrations.
- KMS integration.
- Structured logging.
- Audit events.
- Rate-limit abstraction.
- CI dependency and vulnerability scanning.
- Integration-test infrastructure.

MVP

The MVP should contain:

Identity

- User registration.
- Username and email normalization.
- Argon2id password storage.
- Email verification.
- Account status management.
- Password reset.
- Password change.
- Session creation and revocation.

OAuth/OIDC

- OAuth Authorization Code flow.
- PKCE with S256.
- Exact redirect URI matching.
- Client registration.
- Scopes.
- Consent.
- Token endpoint.
- Token revocation.
- Discovery metadata.
- JWKS endpoint.
- OIDC ID tokens if OIDC is required by clients.

Security

- Login rate limiting.
- Password-reset rate limiting.
- Generic anti-enumeration responses.
- Refresh-token rotation.

- Refresh-token reuse detection.
- CSRF protection for cookies.
- Explicit CORS.
- Audit events.
- Key rotation design.

MFA

- TOTP enrollment.
- TOTP challenge.
- Recovery codes.
- MFA policy enforcement.
- MFA disablement protection.

Do not include LDAP, SAML and every social provider in the first release unless they are unavoidable.

Medium-term improvements

- Google OIDC adapter.
- GitHub and Discord adapters.
- WebAuthn/passkeys.
- Device and session management UI.
- Risk-based authentication.
- Suspicious-login detection.
- Account recovery workflows.
- OIDC federation.
- SCIM provisioning.
- Admin audit UI.
- Tenant-aware authorization.
- Better token exchange and service-account management.
- Provider metadata refresh with strict allowlists.

Enterprise features

- LDAP federation.
- SAML federation.
- Enterprise key management and HSM.
- Private-key JWT client authentication.
- mTLS-bound tokens.
- DPOP.
- PAR and stronger OAuth profiles.
- Fine-grained policy engine.
- Delegated administration.
- Organization and group management.
- SCIM 2.0.
- Regional data residency.
- Advanced audit retention.
- SIEM integration.
- Formal security review and penetration testing.

Recommended final architecture

The most pragmatic target architecture is:

- Ktor modular monolith
- standards-compliant OAuth2/OIDC protocol layer
- typed identity flows
- PostgreSQL source of truth
- Redis for ephemeral distributed state
- Argon2id local credentials
- rotating opaque refresh tokens
- short-lived access tokens
- TOTP MFA
- provider SPI for external identities
- transactional outbox
- centralized audit and rate limiting
- isolated LDAP/SAML/WebAuthn adapters

The most important implementation rule is:

Keep security policy explicit in named flows, keep protocol handling standards-compliant, and make all external side effects asynchronous and auditable.

Files

I created the agent-oriented Markdown prompt basis here:

[Download](#) `identity-auth-provider-agent-prompt.md`

The current runtime does not expose a binary PDF renderer, so I have not fabricated an invalid `.pdf` file. This response is structured as PDF-ready

Markdown and can be exported with:

```
pandoc identity-auth-provider-design.md \  
--from=gfm \  
--toc \  
--number-sections \  
-V geometry:margin=2.2cm \  
-o identity-auth-provider-design.pdf
```

Alternatively, use the browser's **Print → Save as PDF** function on this response.